SIMATS SCHOOL OF ENGINEERING

Approved By AICTE | IET-UK Accreditation

Course Code: DSA03	Course Name: Natural Language Processing
Branch: CSE	Year : 2025

Name: R.Rohith Manikanta
Reg no: 192425326

Assignment 1

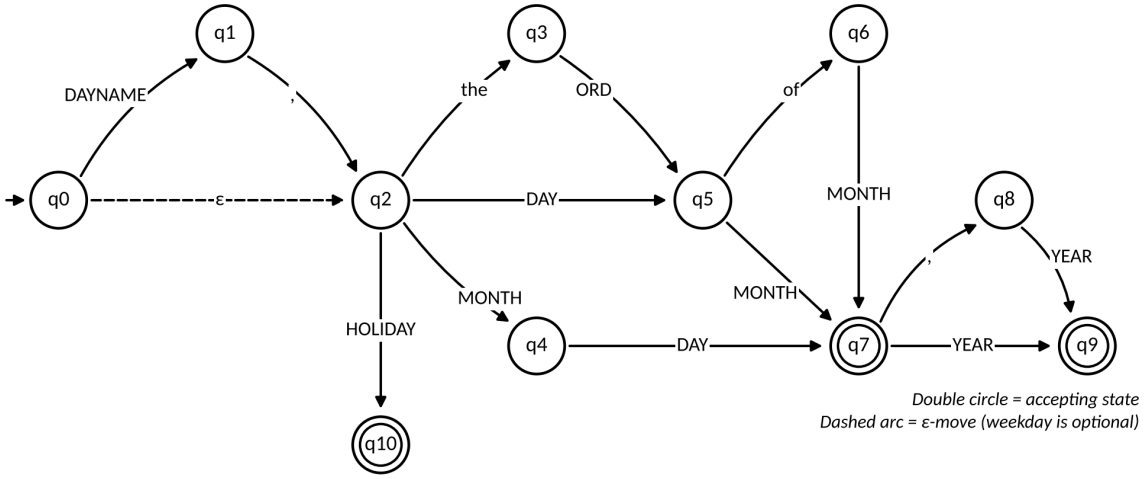
1. Design an FSA that recognizes simple date expressions like March 15, the 22nd of November, Christmas. You should try to include all such "absolute" dates, (e.g. not "deictic" ones relative to the current day like the day before yesterday). Each edge of the graph should have a word or a set of words on it. You should use some sort of shorthand for classes of words to avoid drawing too many arcs (e.g., furniture→ desk, chair, table). (10M) (CO1) (BLT4)
2. Write regular expressions for the following languages: You may use either Perl notation or the minimal "algebraic" notation of Sec. 2.3, but make sure to say which one you are using. By "word", we mean an alphabetic string separated from other words by white space, any relevant punctuation, line breaks, and so forth. (10M) (CO1) (BLT4)
   a. the set of all alphabetic strings.
   b. the set of all lowercase alphabetic strings ending in a b.
   c. the set of all strings with two consecutive repeated words (e.g., "Humbert Humbert" and "the the" but not "the bug" or "the big bug").
   d. the set of all strings from the alphabet a,b such that each a is immediately preceded and immediately followed by a b.
   e. all strings that start at the beginning of the line with an integer and that end at the end of the line with a word.
   f. all strings which have both the word grotto and the word raven in them. (but not, for example, words like grottos that merely contain the word grotto).
   g. write a pattern which places the first word of an English sentence in a register.

   Deal with punctuation.
3. Computing minimum edit distances by hand, figure out whether drive is closer to brief or to

divers, and what the edit distance is. You may use any version of distance that you like. (10M) (CO1) (BLT4)

Answer 1. FSA for absolute date expressions

The automaton below accepts absolute dates such as *March 15*, *the 22nd of November*, *Monday*, *March 15, 2025* and *Christmas*. Every edge carries a word or a class of words; the classes are defined under the figure. q0 is the start state and the double-circled states q7, q9 and q10 are accepting.



MONTH	January | February | … | December, and abbreviations Jan | Feb | Mar | Apr | Jun | Jul | Aug | Sep | Sept | Oct | Nov | Dec (optionally followed by ".")
DAYNAME	Monday | Tuesday | Wednesday | Thursday | Friday | Saturday | Sunday
ORD	1st | 2nd | 3rd | 4th | … | 31st, or first | second | third | … | thirty-first
DAY	ORD or a plain number 1 | 2 | … | 31 (so "15" and "15th" are both accepted)
YEAR	any four-digit number such as 1999, 2025
HOLIDAY	Christmas | Christmas Day | Christmas Eve | New Year's Day | New Year's Eve | Valentine's Day | Halloween | Easter | Thanksgiving | Independence Day | Diwali | Pongal

Sample traces: "March 15" = q0 ε q2 MONTH q4 DAY q7; "the 22nd of November" = q0 ε q2 the q3 ORD q5 of q6 MONTH q7; "Monday, March 15, 2025" = q0 DAYNAME q1 "," q2 MONTH q4 DAY q7 "," q8 YEAR q9; "Christmas" = q0 ε q2 HOLIDAY q10. The ε-arc (q0 to q2) makes the weekday optional, and the shorthand classes keep the figure to 15 arcs instead of hundreds.

1.1 What counts as an "absolute" date

An absolute date names a point on the calendar whose meaning does not change with the moment of speaking. Three families of expressions are covered: (i) named holidays such as *Christmas* or *Diwali*; (ii) day–month forms in either order, such as *March 15*, *15 March* or *the 22nd of November*; and (iii) any of the day–month forms extended by an optional weekday in front (*Monday*, *March 15*) and an optional year at the end (*March 15*, *2025*). Deictic expressions such as *yesterday*, *the day before yesterday*, *next Friday* or *three weeks ago* are deliberately left out: their value depends on the date of the utterance, which a purely lexical automaton does not know, so the machine has no arc for such words and rejects them.

1.2 Formal definition of the machine

The date recognizer is the 5-tuple M = (Q, Σ, δ, q0, F). Here Q = {q0, q1, …, q10}; the alphabet Σ consists of the word classes DAYNAME, MONTH, ORD, DAY, YEAR, HOLIDAY together with the literal tokens *the*, *of* and the comma; q0 is the start state; F = {q7, q9, q10} is the set of accepting states; and δ is the transition relation listed in the table below. Because of the single ε-move from q0 to q2 the machine is strictly a non-deterministic FSA with ε-transitions. This is no problem: the subset construction turns it into an ordinary deterministic automaton whose start state is the set {q0, q2}, so the language it accepts is regular.

1.3 Transition table (all 15 arcs of the figure)

#	From	Input (word or class)	To	What the arc contributes
1	q0	DAYNAME	q1	optional weekday name (Monday, Tuesday, …)
2	q1	","	q2	comma after the weekday
3	q0	ε	q2	weekday skipped (this is what makes it optional)
4	q2	HOLIDAY	q10	Christmas, Diwali, New Year's Eve, …
5	q2	MONTH	q4	month first: "March …"
6	q4	DAY	q7	"… 15" or "… 15th"
7	q2	the	q3	"the …"
8	q3	ORD	q5	"the 22nd", "the twenty-second"
9	q2	DAY	q5	day first: "15 …"
10	q5	of	q6	"… of …"
11	q6	MONTH	q7	"… of November"
12	q5	MONTH	q7	"22nd November" (no "of")
13	q7	","	q8	comma before the year
14	q8	YEAR	q9	", 2025"
15	q7	YEAR	q9	"March 15 2025" (no comma)

1.4 Role of each state

State	Meaning (what the machine has just read)	Accepting?
q0	nothing yet (start state)	No
q1	a weekday name; a comma must follow	No
q2	hub state: the optional weekday part is finished and the date proper begins	No
q3	the word "the"; an ordinal must follow	No
q4	a month name in month-first order; a day is expected	No
q5	a day number in day-first order; "of" or a month is expected	No
q6	the word "of"; a month must follow	No
q7	a complete day–month date; a comma or a year may still follow	Yes
q8	a comma after the date; a year must follow	No
q9	a complete date with a year	Yes
q10	a complete holiday name	Yes

1.5 Further accepted and rejected inputs

Input	Path through the machine	Result
Dec. 25th	q0 ε q2 MONTH q4 DAY q7	Accepted
15 March 2025	q0 ε q2 DAY q5 MONTH q7 YEAR q9	Accepted
Friday, the 3rd of June, 2022	q0 DAYNAME q1 “,” q2 the q3 ORD q5 of q6 MONTH q7 “,” q8 YEAR q9	Accepted
Saturday, Diwali	q0 DAYNAME q1 “,” q2 HOLIDAY q10	Accepted
Christmas Eve	q0 ε q2 HOLIDAY q10 (two-word member of HOLIDAY)	Accepted
the day before yesterday	q2 the q3, then “day” has no arc	Rejected (deictic)
yesterday	no arc from q2 on “yesterday”	Rejected (deictic)
Monday	stops in q1, which is not accepting	Rejected
March	stops in q4, which is not accepting	Rejected
the 22nd	stops in q5, which is not accepting	Rejected
the 22 of November	q3 needs an ordinal; plain “22” is not in ORD	Rejected

1.6 The same language as a regular expression

Since the machine has no loops, its language can be written as one expression over the word classes (spaces between tokens are written as \s+ in a real pattern):

(DAYNAME ,\s+)? (HOLIDAY | MONTH \s+ DAY (,?\s+ YEAR)? | (the \s+ ORD | DAY) \s+ (of \s+)? MONTH (,?\s+ YEAR)?)

The first group is arcs 1–3, the three alternatives are arcs 4, 5–6 and 7–12, and the optional tail (,?\s+ YEAR) is arcs 13–15. Writing it this way is a useful cross-check that the figure and the expression describe the same set of strings.

1.7 Limitations and possible extensions

- **Syntax, not meaning.** The machine accepts “February 31” because DAY allows 1–31 for every month. Rejecting it needs extra memory (a month-to-length table), which goes beyond a finite-state check on word classes.
- **Ambiguity of words.** “May” is both a month and a modal verb, and “Easter” or “Mark” can be names; a tagger or context window is needed before the automaton is used on running text.
- **Other absolute formats.** Numeric forms (15/03/2025, 2025-03-15), forms such as “the Fourth of July”, “Boxing Day”, “the Ides of March”, “mid-March” and era markers (AD, BC) can be added simply by extending the classes or adding arcs.
- **Case and punctuation.** Matching is meant to be case-insensitive, and a trailing full stop after an abbreviation (Dec.) belongs to the MONTH class rather than being a separate token.

Answer 2. Regular expressions (Perl notation is used throughout)

Here “word” means a maximal alphabetic string, so \b (word boundary) is used wherever a pattern has to find words inside running text, and punctuation is treated as a separator.

How to read the notation. In Perl notation a pattern is written between slashes. ^ and $ anchor the pattern to the start and end of the string (or line, with the /m flag); [A-Za-z] is a character class; * means zero or more, + one or more and ? zero or one repetitions; ( ) groups and captures,

(?:) groups without capturing; | is disjunction; \b is a zero-width word boundary and \s is white space; \1 repeats the text captured by the first group. The flags used below are /i (ignore case), /g (all matches) and /m (^ and $ work per line). Every pattern was also run in Python's re module (whose syntax matches Perl for these constructs) on the test strings shown, and the results in the tables are the ones actually produced.

a. All alphabetic strings:

/^[A-Za-z]+$/

(to pick words out of text: /\b[A-Za-z]+\b/)

Breakdown. The class [A-Za-z] stands for one letter, + demands at least one of them, and ^…$ forces the whole string to consist of letters only. In the minimal algebraic notation the same language is [A-Za-z][A-Za-z]*. To cover accented letters in a Unicode setting the class would be replaced by \p{L}.

Test string	Match?	Reason
hello	Yes	letters only
Hello	Yes	upper case is also alphabetic
hello1	No	contains a digit
don't	No	apostrophe is punctuation, i.e. a separator

b. Lowercase alphabetic strings ending in b:

/^[a-z]*b$/

(in text: /\b[a-z]*b\b/; the single string "b" is accepted)

Breakdown. [a-z]* allows any number (including zero) of lowercase letters, and the final b is then fixed by the pattern. The algebraic form is [a-z]*b. Because the star may match nothing, "b" itself belongs to the language, while the empty string does not (the last b is mandatory).

Test string	Match?	Reason
b	Yes	empty prefix, then b
club	Yes	lowercase and ends in b
bat	No	ends in t
Club	No	capital C is not in [a-z]

c. Two consecutive repeated words:

/\b([A-Za-z]+)\s+\1\b/i

The first \b stops a match from starting inside a word, the back-reference \1 repeats the captured word, and the final \b rejects "the theory". \s+ also covers a line break between the two words. It matches "the the" and "Humbert Humbert" but not "the bug" or "the big bug".

Further notes. The group ([A-Za-z]+) captures the first word greedily; if the second word only starts with the same letters ("the theory"), the final \b fails and the engine backtracks, so no false match is reported. The /i flag makes "The the" a hit. A back-reference cannot be written in the minimal algebraic notation, because the language "some word repeated twice" is not regular – this is a case where Perl patterns are strictly more powerful than

regular expressions in the formal sense.

Test string	Match?	Reason
the the	Yes	same word twice
Humbert Humbert	Yes	same word twice
Paris in the the spring	Yes	found inside a longer string
the bug	No	different words
the big bug	No	repeated word is not consecutive

d. Strings over {a,b} where every a has a b immediately before and after it:

/^(b(ab|b)*)?$/

The string must start with b, and after that it is built only from the units “b” and “ab”, so each a is preceded by the b that ends the previous unit and followed by the b inside its own unit. It accepts b, bb, bab, babbab and (vacuously) the empty string, and rejects a, ab, ba, baab.

Check and equivalent automaton. I verified the pattern by brute force against the definition “every a has a b on both sides” for all 8,191 strings over {a,b} of length 0 to 12, with no disagreement. The same language is recognised by a four-state DFA (D is a dead state):

State	on b	on a	Accepting?
S (start, empty so far)	B	D	Yes
B (last symbol was b)	B	A	Yes
A (last symbol was a)	B	D	No
D (dead)	D	D	No

State A is reached only after an a, and it must be followed by b; an a at the very start (state S) or directly after another a leads to D. Ending in A rejects a string whose last symbol is an a, which has no b after it.

Test string	Match?	Reason
b	Yes	no a at all
bab	Yes	a has b on both sides
babbab	Yes	both a’s are surrounded by b
a	No	no b on either side
ab	No	the a has no b before it
baab	No	the two a’s are adjacent

e. Lines that start with an integer and end with a word:

/^[0-9]+\b.*\b[A-Za-z]+$/

(tolerating final punctuation: /^[0-9]+\b.*\b[A-Za-z]+[.,;:!?]*$/)

^ and $ anchor to the start and end of the line, e.g. “25 students enrolled” matches while “25” alone does not.

Breakdown. [0-9]+ is the integer; the \b after it makes sure the whole number was read (“25abc” is not a line starting with an integer); .* skips the middle of the line; \b[A-Za-z]+ is the last word, and $ forces it to be the end of the line. When the text has several lines, the

/m flag lets ^ and $ work on each line. The second version allows a sentence-final mark, so “25 apples.” also counts.

Test string	Match?	Reason
25 students enrolled	Yes	integer first, word last
3 4 five	Yes	starts with 3, ends with “five”
25	No	there is no final word
25 apples.	No / Yes	No with the basic pattern, Yes with the punctuation-tolerant one

f. Strings containing both the word grotto and the word raven (in either order):

/\bgrotto\b.*\braven\b|\braven\b.*\bgrotto\b/

The alternation covers both orders, and \b on each side rejects “grottos” and “ravens”. An equivalent form using look-aheads is /^(?=.*\bgrotto\b)(?=.*\braven\b)/.

Notes. A single pattern “grotto … raven” would miss strings where raven comes first, which is why the two orders are joined by |. The look-ahead version is easier to extend: each (?=.*\bword\b) checks, without consuming text, that the word occurs somewhere, so adding a third word needs one more look-ahead instead of six orderings. Add /i to accept “Grotto” at the start of a sentence, and /s if the two words can be on different lines.

Test string	Match?	Reason
the raven sat in the grotto	Yes	both words, raven first
grotto and raven	Yes	both words, grotto first
grottos and a raven	No	“grottos” is not the word grotto
only a grotto	No	raven is missing

g. Place the first word of each English sentence in a register:

/(?:^|[.!?]["')]*\s+)["'(]*([A-Za-z]+(?:'[A-Za-z]+)?)/g

A sentence starts at the beginning of the text or after “.”, “!” or “?” (plus any closing quote or bracket) and whitespace. Opening quotes or brackets are skipped, and the parentheses around the word capture it into register \1 (Perl: $1). Punctuation is therefore never captured, and contractions such as “Don’t” are kept whole. Abbreviations like “Dr.” can fool it.

Example run. On the text *The cat sat. “Don’t move!” he said. Is it (really) true? Yes. ‘Maybe’ not.* the register receives, in order: The, Don’t, he, Is, Yes, Maybe. Two of these show the limits of a purely surface pattern: “he” is not a real sentence start (a quotation ended with “!” but the sentence continues), and in “Dr. Smith arrived.” the word Smith would be wrongly taken as a new sentence. Requiring a capital letter, [A-Z][A-Za-z]*, removes the first problem (the run then gives The, Don’t, Is, Yes, Maybe) at the price of missing sentences that begin with a number or a lower-case word; the second problem needs a list of abbreviations, for example negative look-behinds such as (?<!\bDr)(?<!\bMr).

Summary of the patterns

Part	Pattern (Perl)	Main idea
a	`/^[A-Za-z]+$/`	character class with +, anchored
b	`/^[a-z]*b$/`	lowercase prefix, mandatory final b
c	`/\b([A-Za-z]+)\s+\1\b/i`	capture a word, repeat it with \1
d	`/^(b(ab|b)*)?$/`	build the string from the units b and ab
e	`/^[0-9]+\b.*\b[A-Za-z]+$/`	integer at line start, word at line end
f	`/\bgrotto\b.*\braven\b|\braven\b.*\bgrotto\b/`	both word orders with \b on each word
g	`/(?:^|[.!?]["')]*\s+)["'(]*([A-Za-z]+(?:'[A-Za-z]+)?)/g`	skip punctuation and capture the first word

Dealing with punctuation. Three habits are used throughout: (1) \b is preferred to \s as a delimiter, because it also stops at commas, full stops and quotation marks; (2) in the line patterns (e and g) the punctuation that may follow the last word, or precede the first one, is listed explicitly in a character class; and (3) apostrophes inside words (don't) are allowed only where a whole contraction should count as one word.

Answer 3. Minimum edit distance: drive vs. brief and drive vs. divers

Version used: Levenshtein distance with insertion cost 1, deletion cost 1 and substitution cost 2 (a substitution counts as a deletion plus an insertion). Each cell D[i,j] = min(D[i-1,j]+1, D[i,j-1]+1, D[i-1,j-1] + (0 if the letters match, else 2)), with D[i,0]=i and D[0,j]=j. Rows are the source **drive** and columns the target.

3.1 Idea of the method

The minimum edit distance between a source string X of length n and a target string Y of length m is the cheapest sequence of insertions, deletions and substitutions that turns X into Y. Trying all sequences would be exponential, but the problem has optimal substructure: the best way to turn the first i letters of X into the first j letters of Y must end with one of only three last steps, and the rest is itself an optimal edit of shorter prefixes. So the cost of every prefix pair can be stored in a table and built up from the corner, which takes O(n·m) time and space; for these 5-letter words that means only 5 × 5 or 5 × 6 cells.

3.2 Recurrence and algorithm

D[i,j] is the distance between X[1..i] and Y[1..j]. The three possible last steps give the three terms of the minimum: a deletion of X[i] (come from the cell above, +1), an insertion of Y[j] (come from the cell on the left, +1), and a match or substitution of X[i] with Y[j] (come from the diagonal cell, +0 when the letters are equal and +2 otherwise). The first row and column are the costs of building or erasing a whole prefix.

```
D[0,0] = 0
for i = 1..n:  D[i,0] = D[i-1,0] + 1        # i deletions
for j = 1..m:  D[0,j] = D[0,j-1] + 1        # j insertions
for i = 1..n:
    for j = 1..m:
        D[i,j] = min( D[i-1,j]   + 1,       # deletion
                      D[i,j-1]   + 1,       # insertion
                      D[i-1,j-1] + (0 if X[i] == Y[j] else 2) )   # match / sub
distance = D[n,m]     # backtrace from D[n,m] to D[0,0] gives the alignment
```

3.3 Worked examples of single cells

- **drive → brief, D[1,1]** (d against b, no match): min(D[0,1]+1 = 2, D[1,0]+1 = 2, D[0,0]+2 = 2) = 2.
- **drive → brief, D[3,3]** (i against i, match): the diagonal gives D[2,2]+0 = 2, which beats D[2,3]+1 = 4 and D[3,2]+1 = 4, so D[3,3] = 2.
- **drive → brief, D[5,5]** (e against f, no match): min(D[4,5]+1 = 6, D[5,4]+1 = 4, D[4,4]+2 = 6) = 4, the final answer.
- **drive → divers, D[2,5]** (r against r, match): D[1,4]+0 = 3 is smaller than D[1,5]+1 = 5 and D[2,4]+1 = 5, so D[2,5] = 3.
- **drive → divers, D[5,6]** (e against s, no match): min(D[4,6]+1 = 5, D[5,5]+1 = 3, D[4,5]+2 = 5) = 3, the final answer.

3.4 Filled tables with the optimal path

The tables below are the ones from the original solution; the light-grey cells now mark the backtrace path from the final cell back to D[0,0], and the dark-grey cell is the distance.

(i) drive → brief : distance = 4

	#	b	r	i	e	f
#	0	1	2	3	4	5
d	1	2	3	4	5	6
r	2	3	2	3	4	5
i	3	4	3	2	3	4
v	4	5	4	3	4	5
e	5	6	5	4	3	4

(ii) drive → divers : distance = 3

	#	d	i	v	e	r	s
#	0	1	2	3	4	5	6
d	1	0	1	2	3	4	5
r	2	1	2	3	4	3	4
i	3	2	1	2	3	4	5
v	4	3	2	1	2	3	4
e	5	4	3	2	1	2	3

Alignments. drive → brief: substitute d→b (2), r, i match, delete v (1), e matches, insert f (1) = **4**. drive → divers: d matches, delete r (1), i, v, e match, insert r (1), insert s (1) = **3**.

3.5 Alignments drawn column by column

Reading the backtrace path of each table gives the following alignments (a dash means “no letter”). The costs add up to the table values.

(i) drive → brief

source	d	r	i	v	e	-
target	b	r	i	-	e	f
operation	sub	match	match	del	match	ins
cost	2	0	0	1	0	1

(ii) drive → divers

source	d	r	i	v	e	-	-
target	d	-	i	v	e	r	s
operation	match	del	match	match	match	ins	ins
cost	0	1	0	0	0	1	1

For drive → brief the only letters that can be kept are r, i and e, but d and v do not appear in the same relative order in brief, so one substitution (cost 2, because d and b are different letters) and the deletion of v are forced, and the extra f has to be inserted. For drive → divers the letters d, i, v and e already form a common subsequence in the right order, so only the r has to be removed and the final "rs" appended.

3.6 Cross-check with unit-cost Levenshtein and a length-normalised view

With the other common cost scheme (every operation costs 1) the two tables change, but the best total for both pairs is 3: for brief it is substitute d→b, delete v and insert f; for divers it is delete r, insert r and insert s. The two words would therefore tie, which is why the substitution-cost-2 version is the one that separates them. A length-based comparison points the same way: dividing the distance by the combined length of the two words gives 4/10 = 0.40 for drive and brief but 3/11 ≈ 0.27 for drive and divers, so divers is also the closer word in relative terms.

Pair	Edit distance (sub = 2)	Edit distance (all cost 1)	Distance / combined length
drive and brief	4	3	4/10 = 0.40
drive and divers	3	3	3/11 ≈ 0.27

Verification. A short program implementing the recurrence of section 3.2 reproduced every cell shown above and returned 4 and 3 with substitution cost 2, and 3 and 3 with unit costs.

Conclusion: drive is closer to divers, with a minimum edit distance of **3**, whereas the distance to brief is 4. (With plain unit-cost Levenshtein both would tie at 3, so the substitution-cost-2 version is the one that separates them.)

Overall summary of the assignment

Question 1 is answered with an 11-state automaton (15 arcs) over word classes that recognizes holidays, month–day and day–month dates, with an optional weekday and an optional year, and rejects deictic expressions. Question 2 gives seven Perl patterns, each checked on matching and non-matching strings, with the punctuation cases handled explicitly. Question 3 computes the minimum edit distances by the dynamic-programming table and shows that, with substitution cost 2, drive is closer to divers (3) than to brief (4).